

Especificação de Requisitos Contemporâneos

[INF-1041] Engenharia de Software

Professor: Juliana Alves Pereira

Entrega: **30/09**

Objetivo

Nesta atividade você vai construir o mesmo sistema duas vezes com o apoio de uma ferramenta de IA (por exemplo: Claude, ChatGPT, Copilot, Cursor ou Gemini). Os prompts são escritos por você. Na primeira vez, você fará um pedido livre, como faria no dia a dia. Na segunda, você escreverá uma especificação de requisitos a partir da descrição do cliente e a incluirá no seu prompt.

Ao final, você vai comparar os dois resultados e refletir sobre como a qualidade dos requisitos influencia o software gerado.

O sistema: controle de estacionamento

Um cliente deseja um sistema simples para controlar a entrada e a saída de veículos do seu estacionamento e calcular o valor a ser pago. Em uma conversa rápida, ele descreveu o que precisa:

Briefing do cliente

“Tenho um estacionamento com 50 vagas. Cobro R\$8 pela primeira hora e R\$5 por hora adicional, e se passar um pouquinho já conta a hora inteira. Quem fica até 15 minutos não paga. Ninguém paga mais de R\$40 por dia e o pernoite é proibido. Funcionamos de 5:00h as 23:59h Quero saber quantas vagas estão livres.”

Restrições técnicas (valem para as duas abordagens):

- Python 3, em um **único arquivo** .py;
- apenas a biblioteca padrão do Python (sem instalar pacotes);
- interface via terminal.
- o sistema deve possuir as funções:
 - `consultar_vagas()`
 - `registrar_entrada(placa,horario)`
 - `registrar_saida(placa,horario)`

Abordagem A: prompt livre

Abra uma **nova conversa** na ferramenta de IA e escreva o seu próprio prompt pedindo o sistema, da forma como você faria naturalmente. Você pode usar as informações do briefing como quiser, sinta-se livre para construir o prompt de forma natural.

Salve o resultado como `estacionamento_a.py` e envie na entrega da atividade.

Abordagem B: prompt com especificação

Preencha o template do **Anexo** (na última página deste documento) a partir do briefing do cliente, aplicando as características de bons requisitos vistas em aula: **atômicos, não ambíguos, verificáveis, sucintos e rastreáveis**.

Quando o briefing for ambíguo, **tome uma decisão e registre-a** na especificação. Não existe uma única interpretação correta, mas a sua precisa estar documentada.

Depois, abra uma **nova conversa** na **mesma ferramenta** usada na Abordagem A e escreva o seu prompt incluindo a especificação preenchida. Assim como na Abordagem A, você pode continuar a conversa para corrigir erros ou pedir ajustes.

Salve o resultado como `estacionamento_b.py`.

Roteiro de testes

Execute cada teste nas duas versões do sistema, sempre com o programa recém-aberto (estacionamento vazio). Para cada teste, marque um resultado: **Passou**, **Falhou** ou **Não testável** (quando o programa não permite executar o teste; por exemplo, se não for possível informar os horários de entrada e saída). Nos casos de falha, anote o que o programa fez.

ID	Passos	Resultado esperado
T1	Consultar vagas livres	50 vagas livres
T2	Registrar entrada da placa ABC1234 às 10:00; consultar vagas livres	49 vagas livres
T3	Entrada de placa às 10:00; saída às 10:15	R\$ 0,00
T4	Entrada de placa às 10:00; saída às 10:16	R\$ 8,00
T5	Entrada de placa às 10:00; saída às 11:01	R\$ 13,00
T6	Entrada de placa às 08:00; saída às 18:00	R\$ 40,00
T7	Entrada de placa às 23:30; saída às 00:45 do dia seguinte	Saída bloqueada, horário saída anterior à entrada
T8	Registrar entrada de ABC1234; registrar nova entrada de ABC1234	Segunda entrada bloqueada, com mensagem
T9	Registrar 50 entradas com placas diferentes; tentar a 51 ^a	Entrada bloqueada, com mensagem
T10	Registrar saída de uma placa que não entrou	Mensagem de erro, sem cobrança

Observação: se a sua especificação (Abordagem B) documentou uma interpretação diferente para alguma regra ambígua do briefing, o resultado será avaliado de acordo com ela.

Durante a atividade, registre

Para **cada** abordagem, salve:

- a ferramenta de IA utilizada (e o modelo);
- os prompts utilizados;
- os códigos gerados;
- os resultados dos testes propostos;

Essas informações serão pedidas no formulário de avaliação.

Avaliação

Depois de concluir as duas abordagens, responda ao formulário de avaliação disponível em:

<https://forms.gle/knchAPAVqVnJvuOL6>

O formulário vai guiar você na comparação entre as duas versões do sistema.

Entregáveis

1. estacionamento_a.py
2. estacionamento_b.py
3. prompt_a.txt
4. prompt_b.txt
5. responda ao formulário de avaliação.

Anexo: template de especificação

```
# Especificação - Controle de Estacionamento

## 1. Objetivo
<Em 1 a 2 frases: qual problema o sistema resolve e para quem>

## 2. Restrições técnicas
- Python 3, arquivo único, apenas biblioteca padrão
- Interface via terminal
- <outras>

## 3. Requisitos funcionais
<Um requisito por linha, com ID, atômico e verificável>
RF-01: O sistema deve exibir a quantidade de vagas livres.
RF-02: O sistema deve...

## 4. Requisitos não funcionais
RNF-01: RNF-01: O sistema deve exibir todas as mensagens de erro em português.
RNF-02: ...

## 5. Critérios de aceitação
<Pelo menos 5, no formato Dado / Quando / Então
(procure por "cenários gherkin" em caso de dúvidas)
CA-01: Dado que ..., quando ..., então ...
CA-02: ...

## 6. Fora do escopo
<O que o sistema NÃO deve fazer>
```